

Interface-Led Architecture (ILA): AIKernel 実行モデルによって検証される AI 時代のソフトウェア開発方法論

Takuya Sogawa

2026-05-20

Contents

Interface-Led Architecture (ILA)	2
AIKernel 実行モデルによって検証される AI 時代のソフトウェア開発方法論	2
Abstract (概要)	2
Keywords	2
1. Introduction (序論)	2
2. Background and Related Work (背景と関連研究)	3
2.1 Contract-First Development	3
2.2 Component-Based Software Engineering	3
2.3 AI-Assisted Software Development	3
2.4 AIKernel	3
3. Definition of Interface-Led Architecture (ILA の定義)	4
3.1 Contract, Invariant, and Interface	4
4. Structural Decomposition Model (構造分解モデル)	4
4.1 Overview	4
5. Interface Decomposition, Unit Composition, and Skeleton Formation	5
5.1 Interface Decomposition (インタフェース分解)	5
5.2 Unit Composition (ユニット構成)	5
5.3 Skeleton Formation (骨格形成)	6
6. Use-Case Driven Contract Testing (ユースケース駆動の契約テスト)	6
7. AI-Assisted Implementation (AI 支援実装)	7
7.1 AI as a Bounded Implementation Agent	7
7.2 Prompt-Embedded Engineering Heuristics	7
7.3 Controlled Generation Boundary	7
8. Metric-Driven Refinement (メトリクス駆動リファインメント)	8
8.1 平均 20 行・最大 100 行ルール	8
8.2 Structural Metrics	8
8.3 Refinement Loop	9
9. Human-AI Role Separation (人間と AI の責務分離)	9
9.1 AI Responsibilities	9
9.2 Human Responsibilities	9
9.3 Architectural Ownership	10
10. Validation Using AIKernel (AIKernel による検証)	10
10.1 AIKernel Overview	10
10.2 Mapping Between ILA and AIKernel	10
10.3 Pipeline Alignment	11
10.4 Context and ROM Externalization	11
10.5 Contract and Provider Model	11
10.6 ReplayLog and Deterministic Boundaries	12
11. Discussion (考察)	12
11.1 言語非依存性と Python への適用可能性	12
12. Limitations (限界)	13
12.1 Interface 数の増加	13
12.2 Architecture 品質への依存	13
12.3 AI Capability Variance	14
12.4 Empirical Validation の不足	14
13. Conclusion (結論)	14

Interface-Led Architecture (ILA)

AIKernel 実行モデルによって検証される AI 時代のソフトウェア開発方法論

Version: v1.0.0

Status: Zenodo-ready manuscript

Author ORCID: <https://orcid.org/0009-0009-7499-2595>

DOI: 10.5281/zenodo.20290614

Primary validation target: AIKernel.NET

License: CC BY 4.0

Abstract (概要)

大規模言語モデル (LLM) の普及により、ソフトウェア開発は、人間が実装を直接記述する工程から、人間が構造を設計し、AI が実装・テスト・改善を支援する工程へと移行しつつある。コード生成、テスト生成、リファクタリング、ドキュメント作成はすでに AI によって高速化されているが、確率的に振る舞う AI を実装主体として組み込む場合、従来のソフトウェア工学だけでは、責務境界、契約、不変条件、アーキテクチャの安定性を十分に維持できない。

本論文では、AI 支援開発時代に適応するソフトウェア開発方法論として **Interface-Led Architecture (ILA)** を提案する。ILA は、実装ではなくインタフェース、契約、責務境界を起点としてソフトウェアを構築する方法論である。システムは、最小責務境界である Interface に分解され、意味的に関連する Interface が Unit として構成され、複数の Unit が Skeleton として接続される。実装は、この Skeleton と Contract によって制約された範囲内で生成・差し替え・改善される。

ILA の中核には、Interface Decomposition、Unit Composition、Skeleton Formation、Contract-First Development、Use-Case Driven Contract Testing、AI-Assisted Implementation、Metric-Driven Refinement、Human-AI Role Separation がある。AI は実装生成、テスト生成、重複削減、メトリクス改善を通じて下限品質を安定化させる。一方、人間は抽象化、境界設計、責務分割、Contract、Invariant、Governance を定義し、上限品質とアーキテクチャの主権を保持する。

本論文では、ILA の検証環境として **AIKernel** を位置づける。AIKernel は、Pipeline、Context、ROM、Contract、Provider、ReplayLog、決定論的执行境界を備えた AI 実行・統治基盤である。ROM は人間の設計意図や Contract を再利用可能な知識として外部化し、ReplayLog は AI による実装修正が既存の Contract を破壊していないことを確認するための再現性基盤として機能する。

なお、ILA は C# / .NET に限定された方法論ではない。Python においても、Protocol、抽象基底クラス、pydantic model、TypedDict、validation rule、Pipeline、DAG などを用いることで、責務境界、Contract、Skeleton を明示できる。したがって ILA は、AI 時代の複数の主要開発環境に適用可能な構造的ソフトウェア開発方法論である。

本論文の中心的主張は、次の一文に要約される。

AI 時代において、実装は確率化される。
しかし、アーキテクチャは決定論的でなければならない。

Keywords

Interface-Led Architecture; AI-assisted software development; Contract-first development; Software architecture; AIKernel; deterministic execution; ReplayLog; ROM; human-AI collaboration; software governance

1. Introduction (序論)

ソフトウェア工学は長い間、人間の理解力、設計力、実装力を前提として発展してきた。オブジェクト指向、レイヤードアーキテクチャ、ドメイン駆動設計、コンポーネント指向設計、マイクロサービスなどは、いずれも人間が構造を理解し、責務を分割し、実装を保守することを前提としている。

しかし、LLM の発展により、この前提は変化しつつある。AI は現在、コード生成、テスト生成、リファクタリング、仕様説明、ドキュメント作成を高い速度で行うことができる。これにより、実装作業の多くは、人間が直接書くものから、AI に生成させ、人間が評価・統治するものへと移行しつつある。

一方で、AI による実装には固有の問題がある。AI は確率的に振る舞うため、同じ目的に対しても生成結果が揺らぐ。また、文脈が不足すると責務境界を誤り、抽象化を過剰または不足させ、冗長な実装や一貫性のない構造を生み出す可能性がある。

この問題は、単に AI の性能不足として扱うべきではない。より本質的には、従来の開発方法論が「確率的な実装主体」を前提としていないことに起因する。

したがって、AI 時代のソフトウェア開発には、次の問いが必要となる。

実装の一部を確率的な AI に委任する場合、ソフトウェア構造はどのように設計されるべきか。

本論文では、この問いに対する方法論として **Interface-Led Architecture (ILA)** を提案する。

ILA は、実装そのものではなく、インタフェースと契約を中心にソフトウェアを構築する。人間はシステムの骨格、責務境界、契約、不変条件を設計し、AI はその境界内で実装、テスト、改善を担う。

この構造により、AI による実装の揺らぎを許容しながらも、システム全体のアーキテクチャを決定論的に維持することが可能となる。

2. Background and Related Work (背景と関連研究)

2.1 Contract-First Development

Contract-First Development は、実装よりも先に契約を定義する開発思想である。Design by Contract、API-First Development、Interface-Based Design などは、いずれも実装前に期待される振る舞いや境界を明示する点で共通している。

ILA はこれらの考え方を継承しつつ、さらに一歩進める。ILA においてインタフェースは、単なる抽象化手段ではない。インタフェースは、責務境界であり、契約境界であり、AI 実装の許可範囲であり、統治対象でもある。

つまり ILA は、契約を「設計補助」ではなく「構造の正典」として扱う。

2.2 Component-Based Software Engineering

コンポーネント指向開発は、システムを再利用可能な単位へ分割する方法論である。責務を分離し、部品を組み合わせるという考え方は、ILA とも共通する。

しかし、従来のコンポーネント指向設計は、多くの場合、人間が実装を安定して維持することを前提としている。コンポーネントの内部実装が AI によって生成・置換・修正される状況は、必ずしも中心的な想定ではなかった。

ILA は、AI による局所的な実装生成と差し替えを前提とする。そのため、実装単位は小さく、責務境界は明確であり、契約は固定されていないなければならない。

2.3 AI-Assisted Software Development

AI 支援開発は、近年急速に実用化が進んでいる。LLM は、仕様からコードを生成し、既存コードを説明し、テストを作成し、リファクタリング案を提示できる。

しかし、現在の AI 支援開発は、しばしば Prompt-Centric である。すなわち、AI の出力品質は、プロンプト、与えられた文脈、直前の会話履歴に強く依存する。

この状態では、短期的な生産性は向上しても、長期的なアーキテクチャ安定性は保証されない。AI が生成したコードが、その場では動作しても、責務境界を曖昧にし、将来的な保守性を低下させる可能性がある。

ILA は、この問題に対して、プロンプトではなく構造を先に固定する。AI は構造を決めるのではなく、構造によって制約された範囲内で実装を行う。

2.4 AIKernel

AIKernel は、AI アプリケーションのための実行・統治基盤である。AIKernel は、LLM を単なる API 呼び出しとして扱うのではなく、文脈、契約、実行境界、Provider、ReplayLog を通じて制御される実行対象として扱う。

AIKernel の特徴は以下である。

- Pipeline による責務の流れ
- Context の外部化
- ROM による設計知識の外部化
- Contract による不変条件の固定
- Provider による実装単位の分離
- ReplayLog による実行再現性
- 決定論的执行境界

これらは、ILA が求める構造的性質と対応している。

3. Definition of Interface-Led Architecture (ILA の定義)

Interface-Led Architecture (ILA) を、次のように定義する。

ILA とは、インタフェースを最小の責務境界としてシステムを分解し、契約によって不変条件を固定し、AI 支援実装を決定論的な構造の内側に閉じ込めるソフトウェア開発方法論である。

ILA において、インタフェースは単なる型定義ではない。インタフェースは、次の複数の意味を持つ。

- 責務境界
- 契約境界
- 実装差し替え境界
- AI 生成境界
- テスト境界
- 統治境界

このため ILA では、実装よりも先にインタフェースを設計する。実装は、インタフェースと契約によって定義された範囲内で生成される。

ILA の基本思想は、次の一文に要約できる。

実装は変化してよい。

しかし、契約と境界は揺らいではならない。

AI は実装を高速に生成できるが、抽象化の妥当性や境界設計の責任を安定して担えるとは限らない。そのため ILA では、人間が境界を設計し、AI がその境界内で実装を担う。

3.1 Contract, Invariant, and Interface

ILA において、Contract、Invariant、Interface は明確に区別される。

Contract は、外部から観測可能な振る舞いの約束である。たとえば、ある Provider がどのような入力を受け取り、どのような結果を返し、失敗時にどのような状態へ遷移するかは Contract に属する。

Invariant は、実行中または状態遷移中に破ってはならない条件である。たとえば、Null response を成功として扱わないこと、失敗時に Fail-Closed へ遷移すること、ReplayLog に必要な情報を保持することは Invariant として扱われる。

Interface は、Contract を実装境界として表現するための構文的な入口である。したがって Interface は Contract そのものではなく、Contract を実装・検証可能な境界へ写像するための構造である。

この区別により、ILA では次の関係が成立する。

```
Invariant
↓
Contract
↓
Interface
↓
Implementation
```

AI は Implementation を生成できる。しかし、Invariant、Contract、Interface を恣意的に変更してはならない。

4. Structural Decomposition Model (構造分解モデル)

4.1 Overview

ILA は、システムを次の階層構造として捉える。

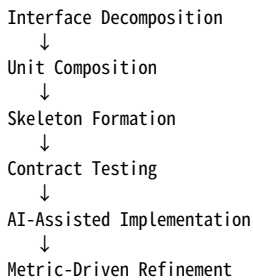
```
System
├── Skeleton
│   └── Units
│       └── Interfaces
│           └── Implementations
```

それぞれの意味は以下である。

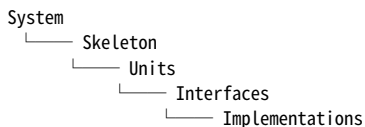
- **Interface:** 最小の責務境界であり、Contract の構文的表現
- **Unit:** 意味的に関連する複数の Interface を束ねた構成単位
- **Skeleton:** 複数の Unit を接続し、システム全体の実行順序、依存方向、統治境界を定義する骨格構造
- **Implementation:** Interface と Contract を満たす差し替え可能な実装

ここで重要なのは、開発順序と完成後の構造階層を区別することである。

ILA における開発順序は次のように表される。



一方、完成後の構造階層は次のように表される。



つまり ILA は、下位の責務境界から設計を始め、上位の構造として組み上げる方法論である。しかし完成したシステムを理解する際には、Skeleton から Unit、Interface、Implementation へと階層的に把握する。

5. Interface Decomposition, Unit Composition, and Skeleton Formation

5.1 Interface Decomposition (インタフェース分解)

ILA における Interface は、最小の責務境界である。Interface は、AI による実装生成、テスト生成、差し替え、検証の基本単位となる。

Interface の特徴は以下である。

- 単一の責務を持つ
- 公開面が小さい
- 独立してテストできる
- 実装を差し替えられる
- AI に実装を委任しやすい
- Contract の構文的表現として機能する

例:

```
public interface IContextProvider
{
    ContextSnapshot Build();
}
```

この例では、IContextProvider は Context を構築する責務だけを持つ。Context の検証、保存、変換、キャッシュなどは別の Interface に分離されるべきである。

ILA では、このような小さな責務境界を多数定義することで、AI が局所的に実装しやすく、人間が統治しやすい構造を作る。

5.2 Unit Composition (ユニット構成)

Unit は、意味的に関連する複数の Interface を束ねた構成単位である。

例:

```
Context Unit
├── IContextProvider
├── IContextValidator
├── IContextSerializer
└── IContextCache
```

Unit は、システム内の一つの責務領域を表す。各 Interface は独立した責務を持つが、Unit によって意味的なまとまりを持つ。

Unit の目的は、単にファイルを整理することではない。責務の近接性を明確にし、依存関係を安定させ、AI が局所的に実装を生成しやすい単位を作ることである。

Unit は、Interface と Skeleton の中間に位置する構造であり、ILA における構成単位を中心である。

5.3 Skeleton Formation (骨格形成)

Skeleton は、システム全体の骨格構造である。これは、Interface や Unit の単なる集合ではなく、実行の流れ、依存方向、統治境界を定義する。

例:

```
[VFS Unit]
↓
[ROM Unit]
↓
[Context Unit]
↓
[Prompt Generation Unit]
↓
[Provider Unit]
↓
[Execution Result Unit]
```

Skeleton は、AI が自由に変更してよい対象ではない。Skeleton は、システムの正典的な構造であり、人間が設計し維持すべき領域である。

AI は Skeleton の内側に配置された Interface の実装を担う。つまり、ILA における AI は、構造の創造者ではなく、構造の内側で動作する実装者である。

6. Use-Case Driven Contract Testing (ユースケース駆動の契約テスト)

ILA では、ユースケースから Contract を導出し、その Contract に基づいてテストを生成する。

基本的な流れは次のとおりである。

```
Use Case
↓
Contract
↓
Test Generation
↓
Implementation
↓
Metric Validation
```

この流れで重要なのは、実装を基準にテストを書くのではなく、ユースケースと Contract を基準にテストを作ることである。

例:

```
Use Case:
Provider は決定論的に検証可能な ExecutionResult を返す。

Contract:
- ExecutionStatus を必ず含む
- Null response を成功として扱わない
- ReplayLog に必要な情報を保持する
- 失敗時は Fail-Closed に遷移する
```

このように Contract を先に固定することで、AI は安全に実装を生成できる。生成された実装が異なっても、Contract を満たしていれば差し替え可能である。

ILA におけるテストは、単なる品質確認ではない。テストは Contract の実行可能な表現であり、AI 実装を統治するための境界である。

7. AI-Assisted Implementation (AI 支援実装)

7.1 AI as a Bounded Implementation Agent

ILA において AI は、制約付きの実装主体である。AI は自由にアーキテクチャを変更する存在ではなく、Interface と Contract によって定義された範囲内で実装を生成する。

AI が担う役割は以下である。

- 実装生成
- テスト生成
- 重複削減
- 冗長なコードの整理
- メトリクス改善
- Boilerplate の削減

一方で、AI が担うべきではない役割も明確である。

- 責務境界の再定義
- Skeleton の変更
- Contract の恣意的変更
- 統治構造の変更
- 不変条件の緩和

この分離により、AI の生成能力を活用しつつ、アーキテクチャの主権を人間が保持できる。

7.2 Prompt-Embedded Engineering Heuristics

ILA では、シニアエンジニアの経験則を Prompt Template として埋め込む。

例:

- One File Per Class / Interface
- Fail-Closed by Default
- Dependency Inversion
- Minimal Public Surface
- Deterministic Naming
- Contract Preservation
- Avoid Hidden Responsibilities

これらは単なるコーディング規約ではない。AI に対して、組織やプロジェクトが持つ設計判断を伝えるための圧縮された知識表現である。

AI は、明示されていない設計意図を安定して推測できるとは限らない。そのため ILA では、シニアエンジニアの判断基準を、繰り返し利用可能なテンプレートとして明文化する。

AIKernel において、この Prompt-Embedded Engineering Heuristics は ROM (Relation-Oriented Markdown) と接続される。ROM は、設計意図、責務境界、Contract、関連する判断理由を、単なる一時的なプロンプトではなく、再利用可能な知識として外部化するための形式である。

これにより、シニアエンジニアの経験則は、その場限りの指示ではなく、AI に注入可能な決定論的知識として扱うことができる。ILA における Prompt Template は、AIKernel 上では ROM によって裏付けられた構造化された設計知識となる。

この構造により、AI は単に「コードを書く」のではなく、明示された設計意図、責務境界、Contract に基づいて実装を生成する。したがって、ROM は ILA における人間の設計判断を AI 実装へ伝達するための媒介として機能する。

7.3 Controlled Generation Boundary

ILA における人間と AI の境界は次のように整理できる。

```
[Human]
Defines:
- Interfaces
- Contracts
- Units
- Skeleton
- Responsibility Boundaries
- Governance Rules
```

- ROM-based Design Knowledge

[AI]

Generates:

- Implementations
- Tests
- Refactorings
- Metric Improvements

この境界により、AI による実装の揺らぎが、システム全体の構造へ波及することを防ぐ。

8. Metric-Driven Refinement (メトリクス駆動リファインメント)

ILA では、メトリクスを用いて実装の状態を継続的に評価する。ここでの目的は、単にコードをきれいにすることではない。

メトリクスは、次のために用いられる。

- 責務肥大化の検知
- 分解不足の検知
- AI が扱いやすい実装単位の維持
- 人間が理解しやすい構造の維持
- Contract 境界の保護

8.1 平均 20 行・最大 100 行ルール

ILA では、実装単位の目安として次を採用する。

- 平均 20 行程度
- 最大 100 行以内

このルールは、機械的な美学ではない。AI と人間の双方にとって、責務を認識しやすい粒度を維持するための経験則である。

実装が長大化する場合、多くの場合そこには次の問題が潜んでいる。

- 複数責務の混在
- 未抽出の Interface
- 不十分な Unit 構成
- 隠れた分岐
- Contract の曖昧化

したがって、長大な実装は単に短くするのではなく、責務境界を見直す契機として扱う。

8.2 Structural Metrics

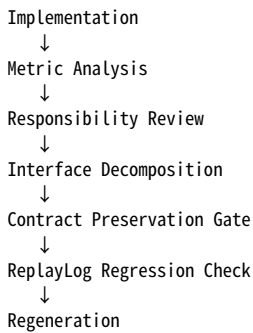
ILA で重視する主なメトリクスは以下である。

Metric	目的
Cyclomatic Complexity	分岐構造の肥大化を検知する
Nesting Depth	認知負荷の増加を検知する
Coupling	依存関係の過密化を検知する
Interface Surface Size	公開面の肥大化を検知する
File / Method Size	責務分離不足を検知する

これらのメトリクスは、単体で絶対的な正しさを示すものではない。しかし、AI による実装生成を継続する環境では、構造劣化を早期に検知するための重要な信号となる。

8.3 Refinement Loop

ILA におけるリファインメントは、次の循環として表される。



重要なのは、メトリクス改善のために **Contract** を壊してはならないという点である。ILA において、改善とは **Contract** を維持したまま、実装と責務境界をより明確にすることである。

Contract Preservation Gate は、AI による修正が既存の **Contract** および **Invariant** を破壊していないかを検証する境界である。AIKernel 上では、この検証は **ReplayLog** に保存された過去の実行文脈、入力、Provider 選択、実行結果を用いたリグレッション確認として実行できる。

この意味で、**ReplayLog** は単なる実行履歴ではない。ILA において **ReplayLog** は、AI による改善が既存の契約を侵食していないことを確認するためのリグレッション検証基盤である。

Metric-Driven Refinement は、実装を小さく、単純に、理解しやすく保つための仕組みである。一方、**ReplayLog** は、その改善が過去の仕様と整合していることを確認するための仕組みである。両者を組み合わせることで、ILA は AI による継続的改善と **Contract Preservation** を両立できる。

9. Human-AI Role Separation (人間と AI の責務分離)

ILA の中核には、人間と AI の責務分離がある。

AI は強力な実装補助者であるが、アーキテクチャの主権者ではない。人間はすべてのコードを書く必要はないが、何を不変とするかを定義しなければならない。

9.1 AI Responsibilities

AI が担うべき領域は、主に下限品質の安定化である。

AI は以下を得意とする。

- テスト生成
- 実装生成
- 重複除去
- コード整形
- メトリクス改善
- 局所的なリファクタリング

これらは、品質の最低ラインを安定させるために有効である。

一方で、AI はプロジェクト全体の目的、責務境界、将来の拡張方向を常に正しく理解できるわけではない。そのため、AI に上位の設計判断を委任することは、構造劣化の原因となり得る。

9.2 Human Responsibilities

人間が担うべき領域は、上限品質の設計である。

人間は以下を定義する。

- 抽象化
- 境界設計
- 責務分割
- Interface

- Unit
- Contract
- Skeleton
- Governance
- 不変条件
- ROM によって外部化される設計知識

特に重要なのは、「何を変えてよいか」ではなく、「何を変えてはならないか」を定義することである。

AI 時代の設計者は、すべてを手で実装する職人ではなく、実装可能な構造を定義する設計者である。

9.3 Architectural Ownership

ILA において、Architecture Ownership は人間に残る。

AI は実装能力を拡張する。しかし、アーキテクチャの正典を定義するのは人間である。

この分離により、次の問題を抑制できる。

- Architecture Collapse
- Prompt-Dependent Drift
- Abstraction Entropy
- Responsibility Leakage
- Contract Erosion

ILA は、人間と AI の協調を前提とするが、その協調は対等な混在ではない。人間が構造を定義し、AI が構造の内側で実装を進める。

10. Validation Using AIKernel (AIKernel による検証)

10.1 AIKernel Overview

ILA は AIKernel に限定された方法論ではない。ILA は、AI 支援開発における構造設計の一般原則である。

一方、AIKernel は、その原則を実行時の文脈管理、契約固定、Provider 分離、ReplayLog によって検証可能にする具体的な実行基盤である。

したがって、本論文における AIKernel の位置づけは、ILA の唯一の実装ではなく、ILA の有効性を示す検証環境である。

AIKernel は、AI アプリケーションのための実行・統治基盤である。LLM を単なる外部 API として扱うのではなく、文脈、契約、Provider、ReplayLog、Governance を通じて制御される実行対象として扱う。

AIKernel の主要な構成要素は以下である。

- Pipeline
- Context
- ROM
- Contract
- Provider
- ReplayLog
- Deterministic Execution Boundary

これらは ILA の構造と対応する。

10.2 Mapping Between ILA and AIKernel

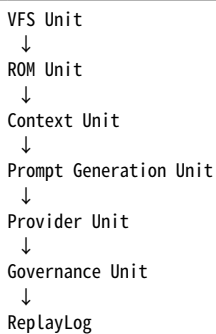
ILA と AIKernel の対応関係は、次のように整理できる。

ILA Concept	AIKernel Mapping
Skeleton	Execution Pipeline
Unit	Context / VFS / Provider / Governance areas
Interface	Provider interfaces / pipeline contracts
Implementation	Concrete providers / pipeline steps
Contract	Invariants and validation rules
ROM	Externalized design knowledge
ReplayLog	Reproducibility and regression boundary

この対応関係により、AIKernel は ILA の抽象理論を実装レベルで検証する環境として機能する。

10.3 Pipeline Alignment

AIKernel の実行構造は、概念的に次のように表せる。



各段階は、明確な責務を持つ。これは ILA における Interface / Unit / Skeleton の考え方と一致する。

Pipeline は、責務の流れを固定する。Context は、暗黙の文脈を外部化する。ROM は、設計意図や Contract に関する知識を正典化する。Contract は、不変条件を固定する。Provider は、実装差し替え単位を提供する。ReplayLog は、実行の再現性を支える。

10.4 Context and ROM Externalization

AIKernel では、Context を暗黙の会話履歴や一時的な Prompt に閉じ込めない。Context は、ROM、Snapshot、DTO、ReplayLog などの形で外部化される。

特に ROM は、ILA における設計知識の外部化に対応する。責務境界、設計意図、Contract、関連する判断理由を ROM として保持することで、AI に渡される文脈は一時的な自然言語指示ではなく、検証可能な知識基盤となる。

これにより、次の効果が得られる。

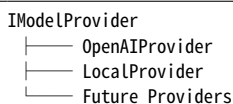
- Hidden State の削減
- 実行文脈の検証可能性
- 設計意図の再利用
- Contract の明示化
- Deterministic Replay
- Governance による制御
- AI 出力の追跡可能性

ILA においても、AI に渡される文脈は明示的でなければならない。AIKernel は、この要求を実行基盤として満たしている。

10.5 Contract and Provider Model

AIKernel における Provider は、ILA における Interface と Implementation の関係として理解できる。

例:



IModelProvider は Interface であり、その背後にある各 Provider は差し替え可能な Implementation である。

Provider は、共通の Contract を満たす限り差し替え可能である。この構造により、AIKernel は特定のモデルやベンダーに依存しない。

ILA の観点では、Provider は次を満たす必要がある。

- Interface によって責務が定義されている
- Contract によって振る舞いが固定されている
- 実装は差し替え可能である

- Pipeline 全体の Skeleton を破壊しない

この性質は、AIKernel が ILA の実践環境として適している理由の一つである。

10.6 ReplayLog and Deterministic Boundaries

AIKernel における ReplayLog は、確率的な AI 実行を検証可能な形に閉じ込めるための重要な構造である。

ReplayLog は以下を保持する。

- 入力文脈
- 実行条件
- Provider の選択
- 実行結果
- 失敗状態
- 再現に必要な情報

これにより、AI の出力が確率的であっても、実行プロセスを検証可能な形で保存できる。

ILA の観点では、ReplayLog は Contract の時間的な証跡である。すなわち、「その時点で、どの Contract のもとで、どの Context に基づき、どの Provider が、どの結果を返したか」を記録する。

ReplayLog は、Metric-Driven Refinement と密接に関係する。AI が実装を短縮し、分解し、複雑度を下げる場合、その変更は必ず Contract を維持していなければならない。AIKernel では、ReplayLog によって過去の実行条件と結果を保存できるため、改善後の実装が既存の仕様を破壊していないかを検証できる。

したがって、AIKernel における ReplayLog は、ILA の Refinement Loop に対して決定論的な安全境界を提供する。これは、AI による実装改善を許可しつつ、仕様の再現性と Contract の継続性を維持するために重要である。

これは、AI 時代のソフトウェアにおける重要な統治機構である。

11. Discussion (考察)

ILA は、ソフトウェア開発の中心を実装から構造へ移す。

従来の開発では、実装能力が品質の中心にあった。優れた開発者は、優れたコードを書き、複雑な実装を理解し、必要に応じて修正する存在だった。

しかし AI 時代には、実装そのものの生成コストは急速に低下する。その一方で、何を実装すべきか、どこに責務を置くべきか、何を不変とすべきかという設計判断の重要性は高まる。

ILA は、この変化に対応する方法論である。

ILA において、ソフトウェア開発の価値は次に移行する。

- 実装量から責務分割へ
- コード生成から Contract 設計へ
- Prompt 技術から構造設計へ
- 局所最適から Skeleton の安定性へ
- 属人的判断から再利用可能な設計規律へ
- 暗黙知から ROM 化された設計知識へ
- 実行結果から ReplayLog による再現可能性へ

この意味で、ILA は AI による開発を否定しない。むしろ、AI を安全に使うために、人間が設計すべき領域を明確化する。

AIKernel は、この ILA の思想を実行基盤として具体化する。ROM によって人間の設計判断を外部化し、ReplayLog によって AI 実装の変更を再検証し、Contract によって不変条件を固定する。これにより、AI は自由な生成主体ではなく、決定論的に統治された実装エージェントとして扱われる。

11.1 言語非依存性と Python への適用可能性

ILA は C# 固有の方法論ではない。本論文では AIKernel.NET との接続を説明するために C# の interface を例示しているが、ILA の本質は特定言語の構文ではなく、責務境界を明示し、Contract によって固定し、AI 実装をその境界内に閉じ込める点にある。

ILA が依拠するのは、言語機能そのものではなく、抽象化、境界、契約、決定論性という普遍的な設計原則である。

Python においても、ILA は十分に適用可能である。C# の interface に相当する責務境界は、Python では `typing.Protocol`、抽象基底クラス (ABC)、または明示的な関数シグネチャとして表現できる。DTO や入出力 Contract は、`dataclasses`、`pydantic model`、`TypedDict`、`validation rule` などによって表現できる。

Python でも Protocol と Contract を明示的に定義することで、動的言語でありながら、C# に近い運用上の構造的保証を再構成できる。ここで重要なのは、型システムの強さそのものではなく、責務境界と入出力契約が明示され、テストと検証によって維持されることである。

また、Unit は Python package、module、service boundary、あるいは feature directory として構成できる。Skeleton は、Pipeline、DAG、workflow graph、dependency graph などとして表現可能である。たとえば、AI エージェント開発では、`retrieval`、`planning`、`tool execution`、`validation`、`memory update` といった処理を Unit として分離し、それらを Pipeline Skeleton として接続できる。

Python における ILA の対応関係は、次のように整理できる。

ILA Concept	Python Mapping
Interface	Protocol / ABC / function signature
Contract	pydantic model / TypedDict / validation rule
Implementation	concrete class / function / callable
Unit	package / module / service boundary
Skeleton	Pipeline / DAG / workflow graph
Metric	complexity / type coverage / test coverage
Replay	test fixture / execution trace / regression log

この対応関係から分かるように、ILA は C# の interface 構文に依存しない。むしろ、Python のような動的言語においてこそ、責務境界と Contract を明示化する意義は大きい。

Python は柔軟性が高い一方で、AI による実装生成では、型境界、入出力仕様、責務分離が曖昧になりやすい。そのため ILA を導入することで、AI が生成する Python コードを、Protocol、pydantic model、テスト、DAG などの構造内に閉じ込めることができる。

したがって ILA は、.NET / C# に限定された設計手法ではなく、Python を含む複数言語に適用可能な、AI 時代のソフトウェア開発方法論である。

12. Limitations (限界)

12.1 Interface 数の増加

ILA はインタフェース粒度の分解を重視するため、インタフェース数が増加しやすい。

これは次の問題を生む可能性がある。

- リポジトリ構造の複雑化
- ファイル数の増加
- ナビゲーション負荷
- 初学者にとっての理解コスト

したがって、ILA を実践するには、命名規則、ディレクトリ構造、ドキュメント、検索性が重要となる。

12.2 Architecture 品質への依存

ILA は、良い分解ができることを前提とする。

誤った責務分割や過剰な抽象化を行うと、ILA は逆に複雑性を増幅する可能性がある。

特に以下には注意が必要である。

- 意味の薄い Interface の乱立
- 実装を持たない抽象の過剰化
- Unit 境界の曖昧化
- Skeleton の肥大化
- Contract の形骸化
- ROM に記録される設計知識の陳腐化

ILA は、単にインタフェースを増やす方法論ではない。責務境界を正しく切るための方法論である。

12.3 AI Capability Variance

AI による実装品質は、利用するモデル、プロンプト、文脈、既存コード品質に依存する。

高性能なモデルであれば、Contract に沿った実装やテスト生成を高い精度で行える。しかし、低性能なモデルや文脈不足の状態では、Contract を誤解したり、責務境界を破ったりする可能性がある。

したがって ILA では、AI の出力を無条件に信頼しない。Contract、Test、Metric、ROM、ReplayLog によって検証可能な形にすることが前提となる。

12.4 Empirical Validation の不足

本論文では AIKernel を検証環境として提示したが、ILA の有効性を広範に示すには、さらなる実証が必要である。

今後の検証対象としては、以下が考えられる。

- 大規模エンタープライズシステム
- 分散システム
- 非.NET 環境
- 複数 AI Provider を用いた実装比較
- メトリクス改善と保守性の相関分析
- ROM による設計知識外部化の有効性評価
- ReplayLog を用いた AI 実装修正の回帰検証

13. Conclusion (結論)

本論文では、AI 支援開発時代に向けたソフトウェア開発方法論として Interface-Led Architecture (ILA) を提案した。

ILA は、インタフェースを最小の責務境界としてシステムを分解し、意味的に関連するインタフェースを Unit として構成し、複数の Unit を Skeleton として接続する。そして Contract によって不変条件を固定し、AI による実装を決定論的な構造の内側に閉じ込める。

ILA の主要な構成要素は以下である。

- Interface Decomposition
- Unit Composition
- Skeleton Formation
- Contract-First Development
- Use-Case Driven Contract Testing
- AI-Assisted Implementation
- Metric-Driven Refinement
- Human-AI Role Separation

また、本論文では AIKernel を ILA の実践的検証環境として示した。AIKernel は、Pipeline、Context、ROM、Contract、Provider、ReplayLog、決定論的実行境界を備えており、ILA の原則と構造的に整合する。

特に ROM は、人間の設計判断、責務境界、Contract を AI に伝達可能な決定論的知識として外部化する。ReplayLog は、AI による実装修正や改善が既存の Contract を破壊していないことを確認するための再現性基盤として機能する。

さらに ILA は C# / .NET に限定された方法論ではない。Python においても Protocol、pydantic model、validation rule、Pipeline、DAG などを用いることで、責務境界、Contract、Skeleton を明示できる。したがって ILA は、AI 時代の主要な開発言語を横断して適用可能なソフトウェア開発方法論である。

本論文の結論は次のとおりである。

AI 時代において、実装は確率化される。しかし、アーキテクチャは決定論的でなければならない。

ILA は、この原則を実践するための構造的なソフトウェア開発方法論である。

References

1. Meyer, Bertrand. Object-Oriented Software Construction. 2nd ed. Prentice Hall, 1997.
2. Parnas, David L. "On the Criteria to Be Used in Decomposing Systems into Modules." Communications of the ACM, 15(12), 1053-1058, 1972.
3. Dijkstra, Edsger W. A Discipline of Programming. Prentice Hall, 1976.
4. Evans, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley, 2003.
5. Martin, Robert C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.

6. Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
7. Brooks, Frederick P. Jr. The Mythical Man-Month: Essays on Software Engineering. Anniversary ed. Addison-Wesley, 1995.
8. Cwalina, Krzysztof, and Brad Abrams. Framework Design Guidelines: Conventions, Idioms, and Patterns for Reusable .NET Libraries. Addison-Wesley, 2005.
9. OpenAI. "GPT-4 Technical Report."arXiv:2303.08774, 2023.
10. Chen, Mark, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, et al. "Evaluating Large Language Models Trained on Code."arXiv:2107.03374, 2021.
11. Amershi, Saleema, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, et al. "Guidelines for Human-AI Interaction."Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, 2019.
12. AIKernel Project Documentation. AIKernel.NET Architecture and Governance Documentation. AIKernel.NET Project, 2026.